DATA WRANGLING LAB EXERCISES

Topic: Handling Missing Values, Inconsistent Data, and Duplicate Records

Question: Create two Pandas DataFrames containing movie details and movie ratings. Include missing values, inconsistent data, and duplicate records. Perform the following operations using Python and Pandas.

DataFrame 1: Movie Details — Movie_ID, Movie_Name, Genre, Release_Year

DataFrame 2: Movie Ratings — Movie_ID, Rating, Number_of_Reviews, Language

Part A: Creating and Inspecting DataFrames

1. Import Pandas, NumPy, and the necessary libraries.

2. Create both DataFrames with at least 10 records each.

3. Include missing values (NaN, None) in numerical and categorical columns.

4. Include inconsistent data, such as “Action”, “action”, and “ACTION” in the Genre column.

5. Include duplicate rows and repeated Movie_ID values.

6. Display both DataFrames using head(), tail(), and info().

7. Generate descriptive statistics using describe().

8. Identify the number of rows and columns using shape.

Part B: Handling Missing Values

9. Identify missing values using isnull() and isna().

10. Count missing values in each column using isnull().sum().

11. Calculate the percentage of missing values in each column.

12. Display only the rows containing at least one missing value.

13. Remove rows containing missing values using dropna().

14. Remove columns containing missing values using dropna(axis=1).

15. Fill missing categorical values with a constant such as “Unknown”.

16. Fill missing numerical values with 0.

17. Replace missing numerical values with the mean of the respective column.

18. Replace missing numerical values with the median of the respective column.

19. Replace missing categorical values with the mode of the respective column.

20. Fill missing values using forward fill (ffill) and backward fill (bfill).

21. Compare the results of mean, median, and mode imputation.

Part C: Using Imputers

22. Apply SimpleImputer from sklearn.impute to handle missing numerical values using the mean strategy.

23. Apply SimpleImputer using the median strategy.

24. Apply SimpleImputer to categorical columns using the most frequent strategy.

25. Apply SimpleImputer using the constant strategy to replace missing values with “Unknown”.

26. Create a separate DataFrame and apply KNNImputer to fill missing numerical values using the nearest-neighbour method.

27. Compare the results obtained using SimpleImputer and KNNImputer.

28. Explain why categorical columns generally require a categorical imputation strategy or suitable encoding before numerical imputation.

Part D: Handling Inconsistent Data

29. Identify inconsistent values in the Genre column.

30. Standardize text using str.lower(), str.upper(), or str.title().

31. Remove leading and trailing spaces using str.strip().

32. Replace inconsistent genre names, such as “Sci-Fi”, “Sci Fi”, and “Science Fiction”, with a standard value.

33. Convert column values to appropriate data types using astype() and pd.to_numeric().

34. Identify invalid ratings outside the permitted range of 0 to 5.

35. Identify invalid release years and review counts.

36. Correct invalid values using suitable replacement rules or mark them as missing.

37. Convert dates to a consistent format using pd.to_datetime(), if a release-date column is included.

Part E: Handling Duplicate Records

38. Identify duplicate rows using duplicated().

39. Count the total number of duplicate rows.

40. Display all duplicate records.

41. Remove duplicate rows using drop_duplicates().

42. Remove duplicates based only on Movie_ID.

43. Retain the first occurrence of a duplicate record.

44. Retain the last occurrence of a duplicate record.

45. Explain why repeated Movie_ID values do not always indicate duplicate records, such as when a movie has multiple language-specific ratings.

Part F: Merging and Cleaning DataFrames

46. Merge the two DataFrames using Movie_ID.

47. Compare inner, left, right, and outer joins.

48. Identify missing values introduced by the merge.

49. Check whether Movie_ID is unique in each DataFrame before merging.

50. Handle duplicate keys appropriately to avoid unintended row multiplication.

51. Apply suitable imputation strategies to the merged DataFrame.

52. Standardize inconsistent text values and remove duplicate records.

53. Verify the cleaned DataFrame using isnull().sum(), duplicated().sum(), and info().

Part G: Final Data Analysis and Export

54. Display movies with a rating greater than 4.

55. Sort movies by rating in descending order.

56. Display the movie with the highest rating.

57. Calculate the average rating for each genre using groupby().

58. Save the cleaned DataFrame to cleaned_movies.csv.

59. Read the saved CSV file back into Pandas and verify the data.

60. Write a short conclusion comparing the dataset before and after cleaning.

Libraries and Functions to Practise

pandas, numpy, sklearn.impute.SimpleImputer, KNNImputer, isnull(), fillna(), dropna(), duplicated(), drop_duplicates(), replace(), astype(), pd.to_numeric(), pd.to_datetime(), merge(), and groupby().

Important Note on Machine Learning

Fit imputers only on the training data when preparing data for machine learning. Use the fitted imputers to transform validation and test data to help prevent data leakage.